Day 74: Self-Attention, From Scratch — Attention Within One Sequence

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 5 | Complete study guide

Watch: The math behind Attention: Keys, Queries, and Values matrices (Luis Serrano Academy) | L19.4.2 Self-Attention and Scaled Dot-Product Attention (Sebastian Raschka)

Script: `day74_self_attention.py` · every number in this guide is copied by program from `day74_run_output.txt`, the real output of that script.

So far a decoder attended to an encoder. In **self-attention** every position of one sequence attends to every position of the same sequence. Built from scratch and put on Week 9's real part-of-speech data, a single self-attention layer lifts test accuracy from 0.7726 (the same model with the attention removed) to 0.8070, and more than doubles the accuracy on unknown words. It stays below the Week 10 recurrent taggers, and the shuffle test shows why: self-attention as built today does not see word order at all. Shuffling a sentence's words changes none of its predictions.

0. Where today fits

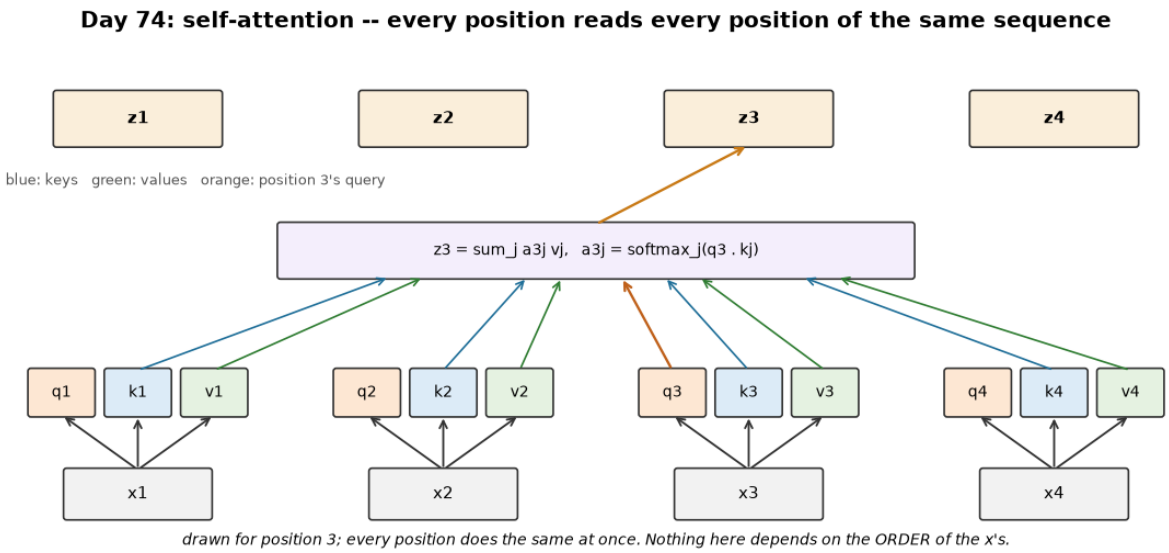
DEFINITION — start here, no prior knowledge assumed

Self-attention takes a sequence of vectors $x_1, \ldots, x_n$ and makes three projections of each: a **query** $q_i = x_i W_q$, a **key** $k_i = x_i W_k$ and a **value** $v_i = x_i W_v$. Position i scores every key with its query, turns the scores into weights with a softmax, and outputs the weighted average of the values. Every position does this at once.

REAL-WORLD SCENARIO

A meeting where everyone may listen to everyone: each person decides whom to pay attention to (query against the others' keys), then takes away a blend of what those people said (values). But if the seating order is shuffled, nobody notices: the rule never refers to where anyone sits.

SYSTEM DIAGRAM



Drawn for position 3: its query is scored against every key; the weights mix every value into z3. All positions do the same at once.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One self-attention layer, by hand

DEFINITION — start here, no prior knowledge assumed

Three positions with two numbers each, the projections chosen by hand so every number can be followed.

REAL-WORLD SCENARIO

Working out who listens to whom in a three-person meeting on paper.

MATHEMATICS

$$Q = X\,W_q,\quad K = X\,W_k,\quad V = X\,W_v$$

$$S = Q\,K^T\quad (S_{ij} = q_i \cdot k_j);\quad A = \text{row-wise softmax}(S);\quad Z = A\,V$$

unscaled today; Day 75 divides S by $\sqrt{d}$

SPECIFICATION

Inputs x = (1, 0), (0, 1), (1, 1); $W_q = W_k$ = identity; W_v = [[1, 0], [0, 2]].

Why this test

Every position attends to itself as well as the others; the hand computation makes that visible.

IMPLEMENTATION

```
def self_attention(X, Wq, Wk, Wv, mask):
    Q, K, Vv = X @ Wq, X @ Wk, X @ Wv
    S = np.einsum("mid,mjd->mij", Q, K)
    A = masked_softmax(S, mask[:, None, :])
    Z = np.einsum("mij,mjd->mid", A, Vv)
    return Z, dict(X=X, Q=Q, K=K, V=Vv, A=A)
```

ACTUAL OUTPUT

```
scores by hand (row i = position i's query against every key):
[[1. 0. 1.]
 [0. 1. 1.]
 [1. 1. 2.]]
weights by hand:
[[0.4223 0.1554 0.4223]
```

```
 [0.1554 0.4223 0.4223]
 [0.2119 0.2119 0.5761]]
output by hand A (X Wv):
[[0.8446 1.1554]
 [0.5777 1.6893]
 [0.7881 1.5761]]
largest difference from self_attention: 0.00e+00
Actual output from this lesson's run.
```

What it means

The function matches the hand arithmetic exactly (0.00e+00). Position 3, x = (1, 1), overlaps with both others and with itself most, so it puts 0.5761 of its weight on itself. Its output is a blend of all three values.

WHY CHOOSE THIS?

Every position gets context from the whole sequence in one step, with no recurrence.

WHY NOT THIS? (TRADEOFFS)

Each position also attends to itself; nothing forces it to look at others.

2. Self-attention ignores order

DEFINITION — start here, no prior knowledge assumed

Permutation equivariance: if the positions are shuffled, the outputs are shuffled in exactly the same way, and nothing else changes. A model with this property cannot tell one order of its inputs from another.

REAL-WORLD SCENARIO

The meeting again: reseat everyone and each person still hears the same blend, because the rule never mentions seats.

SPECIFICATION

Data	4 random sequences of 7 positions with 5 numbers each, random projections, one random shuffle.

Why this test

It shows, in one number, the most important property of self-attention without positional information.

IMPLEMENTATION

```
Z, _ = self_attention(X, Wq, Wk, Wv, mask)
perm = rng.permutation(7)
Zp, _ = self_attention(X[:, perm], Wq, Wk, Wv, mask)
```

ACTUAL OUTPUT

```
shuffle [3, 5, 4, 1, 6, 0, 2]: largest |output(shuffled) - shuffled(output)| = 1.33e-15
for comparison, outputs themselves are of size 1.715 on average
Actual output from this lesson's run.
```

What it means

Shuffled inputs give exactly the shuffled outputs, up to rounding (1.33e-15 against outputs of size 1.715). Self-attention treats a sequence as a **set**. The RNNs of Weeks 9–10 read in order; this layer does not.

WHY CHOOSE THIS?

Every pair of positions is connected directly, however far apart.

WHY NOT THIS? (TRADEOFFS)

Order is invisible unless it is added to the inputs (positional encoding, Day 77).

3. Gradient check, with padding

DEFINITION — start here, no prior knowledge assumed

Sentences in a batch have different lengths. Padded positions must get zero weight as keys, through Day 70's masked softmax, and no gradient.

REAL-WORLD SCENARIO

Empty chairs at the meeting: nobody should listen to them.

MATHEMATICS

$$dA = dZ\,V^T,\quad dV = A^T\,dZ;\quad dS = A \times (dA - \Sigma_j A_{ij}\,dA_{ij})$$

$$dQ = dS\,K,\quad dK = dS^T\,Q;\quad dX = dQ\,W_q^T + dK\,W_k^T + dV\,W_v^T$$

SPECIFICATION

Model	The tagger of Section 4 at tiny size: embedding 4, attention 3, hidden 5, 12 words, 3 tags.
Data	Day 62's four sentences of lengths 5, 2, 4, 1; one word repeated; Day 62's pass rule.

Why this test

X makes queries, keys and values, so its gradient has three parts; the mask adds a fourth way to go wrong.

IMPLEMENTATION

```
def self_attention_backward(dZ, cache, Wq, Wk, Wv):
    X, Q, K, Vv, A = (cache[k] for k in ["X", "Q", "K", "V", "A"])
    dA = np.einsum("mid,mjd->mij", dZ, Vv)
    dV = np.einsum("mij,mid->mjd", A, dZ)
    dS = A * (dA - (A * dA).sum(axis=2, keepdims=True))
    dQ = np.einsum("mij,mjd->mid", dS, K)
    dK = np.einsum("mij,mid->mjd", dS, Q)
    dX = dQ @ Wq.T + dK @ Wk.T + dV @ Wv.T
    flat = lambda T: T.reshape(-1, T.shape[-1])
    return dX, flat(X).T @ flat(dQ), flat(X).T @ flat(dK), flat(X).T @ flat(dV)
```

ACTUAL OUTPUT

```
 parameter  entries  max relative error  max |difference|   pass
         E       48            1.77e-08          2.19e-10   True
        Wq       12            4.57e-07          1.01e-11   True
        Wk       12            5.36e-06          1.68e-11   True
        Wv       12            3.07e-07          3.43e-11   True
        W1       35            3.48e-08          8.01e-11   True
        b1        5            1.02e-07          1.35e-11   True
       Why       15            6.52e-09          3.91e-11   True
        by        3            1.10e-08          1.96e-11   True
every entry passes: True;  <pad>'s embedding gradient exactly zero: True;  largest weight on a
    padded key: 0.0e+00
Actual output from this lesson's run.
```

What it means

Every entry passes, padding's embedding row gets no gradient, and the largest weight any query puts on a padded key is exactly 0.0e+00.

WHY CHOOSE THIS?

The full model is correct before it is trained.

WHY NOT THIS? (TRADEOFFS)

A tiny model; the trained one is larger but runs the same code.

4. On Week 9's real tagging task

DEFINITION — start here, no prior knowledge assumed

The tagger: word vectors (embedding 32) → self-attention (size 32) → a hidden layer on [the word's own vector ; its attention output] → a tag for every word. The control is the identical model with the attention output replaced by zeros: each word tagged on its own.

REAL-WORLD SCENARIO

Tagging each word after hearing every other word of the sentence, in no particular order, against tagging each word with no context at all.

SPECIFICATION

Training	Day 67's setup: Day 62's data and splits, batch 32, clip 1.0, 25 epochs; learning rate chosen on validation from 0.5, 1, 2, 4 (seed 0).
References	Day 62: most-frequent-tag 0.8016, RNN 0.8327; Day 67: LSTM 0.8377; Day 68: bidirectional LSTM 0.8577.

Why this test

It measures what one attention layer adds on real text, against a control that differs only by the attention.

IMPLEMENTATION

```
def forward(self, ids):
    m, T = ids.shape
    self.ids, self.mask = ids, ids != PAD
    self.X = self.E[ids]
    if self.attend:
        self.Z, self.cache = self_attention(self.X, self.Wq, self.Wk, self.Wv, self.mask)
    else:
        self.Z = np.zeros((m, T, self.Wv.shape[1]))
    self.XZ = np.concatenate([self.X, self.Z], axis=2).reshape(m * T, -1)
    self.Hh = np.tanh(self.XZ @ self.W1 + self.b1)
    self.probs = softmax(self.Hh @ self.Why + self.by)
    return self.probs
```

ACTUAL OUTPUT

```
   lr  best val  at epoch  train at end
  0.5    0.7945        21        0.8710
  1.0    0.7899        21        0.8808
  2.0    0.7976        18        0.9049
  4.0    0.7884         6        0.9271
chosen on validation: lr 2.0
model                        seed 0   seed 1   seed 2     mean
self-attention               0.8075   0.8080   0.8056   0.8070
no attention (control)       0.7717   0.7719   0.7743   0.7726
Actual output from this lesson's run.
```

What it means

Self-attention adds context that helps: 0.8070 against 0.7726 for the control, consistent over all three seeds. But it does not reach the recurrent taggers: Day 62's RNN reached 0.8327 and Day 68's bidirectional LSTM 0.8577. Both of those know which word comes before which; this layer does not (Section 5).

WHY CHOOSE THIS?

Context from the whole sentence in one parallel step.

WHY NOT THIS? (TRADEOFFS)

Without word order, it is a bag-of-words context.

5. The shuffle test

DEFINITION — start here, no prior knowledge assumed

Day 62's test: shuffle the words of every test sentence (each keeps its own tag) and tag again.

REAL-WORLD SCENARIO

Reading a sentence with its words in random order: an order-blind reader gives exactly the same answers.

SPECIFICATION

Data	Every test sentence with its real words shuffled (Day 62's shuffle); seed 0 of each model.

Why this test

Day 62's RNN lost accuracy when words were shuffled (0.8327 to 0.7487), because it uses order.

IMPLEMENTATION

```
p, p_sh = net.predict(ids_te), net.predict(ids_sh)
back = np.empty_like(p_sh)
np.put_along_axis(back, order, p_sh, axis=1)
changed = (back != p)[mask_te].mean()
```

ACTUAL OUTPUT

```
model                       normal order  shuffled  predictions changed
self-attention                    0.8075    0.8075               0.0000
no attention (control)            0.7717    0.7717               0.0000
Actual output from this lesson's run.
```

What it means

Shuffling changes **none** of the self-attention tagger's predictions (0.0000): its accuracy is 0.8075 either way. Section 2's property, confirmed on real sentences. Everything it gains over the control comes from which words are in the sentence, not where they are.

WHY CHOOSE THIS?

A clean diagnostic of order-blindness.

WHY NOT THIS? (TRADEOFFS)

It shows what is missing; it does not supply it (Week 12 does).

6. What each word attends to

DEFINITION — start here, no prior knowledge assumed

The trained tagger's weights on every test sentence: the weight a word puts on itself and on words at each distance, against an even spread over the sentence; and the words that receive the most weight.

REAL-WORLD SCENARIO

Watching whom people at the meeting actually listen to.

SPECIFICATION

Model	Seed 0, all test sentences; words seen at least 50 times for the ranking.

Why this test

An order-blind model can still prefer some words; this shows which.

IMPLEMENTATION

```
received = (A * mask_te[:, :, None]).sum(axis=1)
total, count = np.zeros(V), np.zeros(V)
np.add.at(total, ids_te[mask_te], received[mask_te]); np.add.at(count, ids_te[mask_te], 1)
common = np.where(count >= 50)[0]
share = total[common] / count[common]
```

ACTUAL OUTPUT

```
 distance  mean weight  if spread evenly
        0       0.0656            0.0892
        1       0.0875            0.0762
        2       0.0708            0.0684
        3       0.0642            0.0625
        5       0.0550            0.0548
       10       0.0415            0.0421
most attended words (weight received from the whole sentence; an even spread gives 1.00):
  'be'          1.81  (110 occurrences)
  'there'       1.62  (64 occurrences)
  'good'        1.54  (76 occurrences)
  'at'          1.50  (84 occurrences)
  "n't"         1.46  (60 occurrences)
  'like'        1.37  (53 occurrences)
  'great'       1.35  (70 occurrences)
  ':'           1.27  (62 occurrences)
median over these 56 common words: 0.96
Actual output from this lesson's run.
```

What it means

Words put **less** weight on themselves than an even spread would (0.0656 against 0.0892): probably because the word's own vector already reaches the hidden layer directly, so attention is more useful spent on the others (not tested here). Weight changes little with distance, as expected for an order-blind model (the small excess at distance 1, 0.0875 against 0.0762, can only come from which words tend to stand next to each other, since the model cannot see distance). The most attended words are short function words and evaluative words such as 'be' (1.81 times an even share), 'there' and 'good'.

WHY CHOOSE THIS?

Shows what the model reads from its context.

WHY NOT THIS? (TRADEOFFS)

Weights show where the model reads, not how it uses what it reads.

7. Where the context helps

DEFINITION — start here, no prior knowledge assumed

Day 62's word groups: unambiguous, ambiguous and unknown test words.

REAL-WORLD SCENARIO

Which questions get easier once the student is allowed to read the rest of the paragraph.

SPECIFICATION

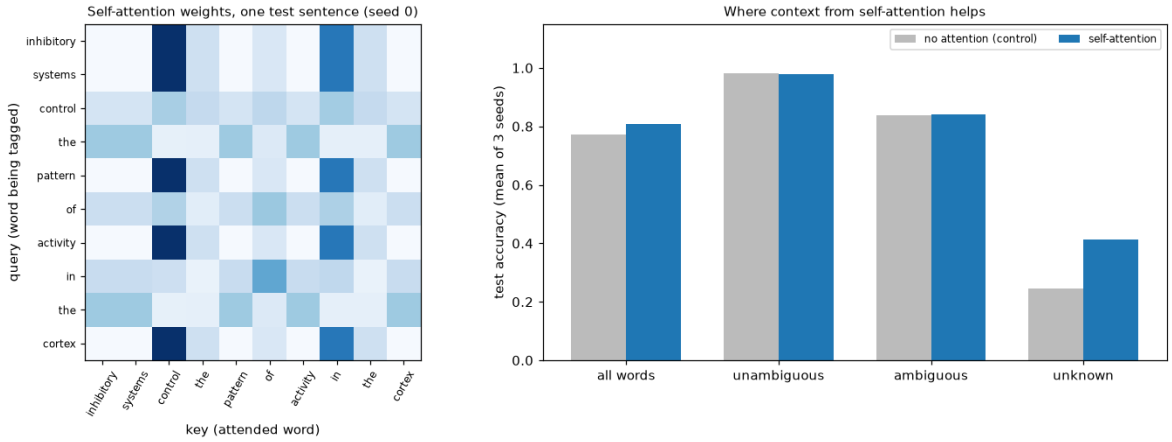
Models	Both taggers, three seeds each.

IMPLEMENTATION

```
groups = {"all words": mask_te, "unambiguous": mask_te & (ids_te != UNK) & ~ambiguous[ids_te],
          "ambiguous": mask_te & ambiguous[ids_te], "unknown": mask_te & (ids_te == UNK)}
```

ACTUAL OUTPUT

```
model                        all words  unambiguous    ambiguous      unknown
self-attention                  0.8070       0.9780       0.8424       0.4140
no attention (control)          0.7726       0.9814       0.8391       0.2440
Actual output from this lesson's run.
```



Left: one test sentence's attention weights (rows: the word being tagged). Right: accuracy by word group.

What it means

The gain is almost all on **unknown** words: 0.4140 against 0.2440. A word the model has never seen can only be tagged from its company, and self-attention supplies that company. On known words the context changes little (0.8424 against 0.8391 on ambiguous words).

WHY CHOOSE THIS?

Context where it matters most: words the model does not know.

WHY NOT THIS? (TRADEOFFS)

Little help on ambiguous words, where order often decides.

8. What it costs

DEFINITION — start here, no prior knowledge assumed

Seconds per training epoch, in this run, for the two taggers.

REAL-WORLD SCENARIO

Listening to everyone takes longer than listening to no one.

MATHEMATICS

work per sentence: $n \times n$ scores and an $n \times n$ weighted average, all computed at once rather than one step after another

ACTUAL OUTPUT

```
self-attention                0.71 s per epoch
no attention (control)        0.16 s per epoch
ratio 4.38
Actual output from this lesson's run.
```

What it means

Self-attention made an epoch 4.38 times as long as the control here. Unlike an RNN its work does not have to wait for the previous word, so on parallel hardware it runs far faster per sentence; this NumPy code runs on one thread.

WHY CHOOSE THIS?

Parallel over positions: no step waits for the previous one.

WHY NOT THIS? (TRADEOFFS)

Work grows with the square of the length.

Interview preparation — technical questions

Q: Write self-attention.

A: Q = X Wq, K = X Wk, V = X Wv; A = softmax(Q K^T) row by row (scaled by sqrt(d) in practice); Z = A V.

Q: How do you handle padding in self-attention?

A: Mask the padded keys: set their scores to minus infinity before the softmax, so their weights are exactly 0.

Q: Is self-attention permutation-equivariant? How would you check?

A: Yes, without positional information: shuffle the positions and compare; here the difference was 1.33e-15.

Q: What does the input's gradient consist of?

A: Three parts, because X made the queries, the keys and the values.

Q: How many scores does one self-attention layer compute for an n-word sentence?

A: n x n, one per pair of positions.

Interview preparation — theoretical questions

Q: Why did the self-attention tagger fall short of the LSTM taggers?

A: It cannot see word order; the shuffle test changed none of its predictions.

Q: What is the fix?

A: Add position information to the inputs: positional encoding (Week 12).

Q: Self-attention versus an RNN?

A: Self-attention connects any two positions in one step and runs in parallel; an RNN is sequential but has order built in.

Q: Why did attention help unknown words most?

A: Their own vector carries no information, so all evidence must come from context.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Self-attention gives every position a view of the whole sequence in one parallel step: queries, keys and values are projections of the same inputs. On real tagging it adds useful context, mostly for unknown words, but it is exactly blind to order: shuffling a sentence changes none of its predictions. That blindness is why it fell short of the recurrent taggers, and why the Transformer adds positional encoding.

- Q, K, V = X Wq, X Wk, X Wv; A = softmax(Q K^T); Z = A V.
- Shuffle in, shuffle out: 1.33e-15 difference. Self-attention sees a set.
- Real tagging: 0.8070 against 0.7726 without attention; unknown words 0.4140 against 0.2440.
- Below the RNN (0.8327) and the bidirectional LSTM (0.8577), which see order.
- Shuffled sentences: 0 predictions changed.
- Cost grows with n x n but runs in parallel.

Suggested watch order, if starting from zero

- 1. The math behind Attention: Keys, Queries, and Values matrices (Luis Serrano Academy) — keys, queries and values, built up slowly.
- 2. L19.4.2 Self-Attention and Scaled Dot-Product Attention (Sebastian Raschka) — self-attention as a lecture, including the scaling (Day 75).

Day 75 preview

- **Scaled dot-product attention.** Today's scores were plain q · k. Day 75 divides them by √d and measures, step by step, what the scaling changes: the scores, the softmax, the gradient and training.
- Today's exercise script, `day74_exercise.py`, isolates one self-attention layer, forward and backward, with padding, and checks that shuffling the input shuffles the output.